

Allocations and compute scheduling

Parent page: [Job scheduling policies](#)

Contents

Allocations for high-performance computing

[From compute allocations to job scheduling](#)

[Consequences of overusing a CPU or GPU allocation](#)

Reference GPU Units (RGUs)

[Choosing GPU models for your project](#)

[RAC awards hold RGU values constant](#)

Detailed effect of resource usage on priority

[Cores equivalent used by the scheduler](#)

[Reference GPU unit equivalent used by the scheduler](#)

[Ratios in bundles](#)

Viewing resource usage in the portal

[Displaying a specified account](#)

[Options in the figure legend](#)

[Displaying the allocation target and queued resources](#)

[Mouse hover over the figure window](#)

[Default SLURM Raw Shares and the SLURM Raw Usage](#)

[Selecting a specific cluster and time interval](#)

[Displaying usage over an extended time period into the future](#)

[Metrics, summation, and running jobs](#)

[Display of the running total of account usage](#)

Viewing resource usage in CCDB

[Usage by compute resource](#)

[Usage by resource allocation project](#)

[GPU usage and Reference GPU Units \(RGUs\)](#)

[Usage by submitter](#)

Allocations for high-performance computing

An allocation is an amount of resources that a research group can target for use for a period of time, usually a year. This amount is either a maximum amount, as is the case for storage, or an average amount of usage over the period, as is the case for shared resources like computation cores.

Allocations are usually made in terms of core years, GPU years, or storage space. Storage allocations are the most straightforward to understand: research groups will get a maximum amount of storage that they can use exclusively throughout the allocation period. Core year and GPU year allocations are more difficult to understand because these allocations are meant to capture average use throughout the allocation period---typically meant to be a year---and this use will occur across a set of resources shared with other research groups.

The time period of an allocation when it is granted is a reference value, used for the calculation of the average which is applied to the actual period during which the resources are available. This means that if the allocation period was a year and the clusters were down for a week of maintenance, a research group would not be entitled to an additional week of resource usage. Equally so, if the allocation period were to be extended by a month, research groups affected by such a change would not see their resource access diminish during this month.

It should be noted that in the case of core year and GPU year allocations, both of which target resource usage averages over time on shared resources, a research group is more likely to hit (or exceed) its target(s) if the resources are used evenly over the allocation period than if the resources are used in bursts or if use is put off until later in the allocation period.

From compute allocations to job scheduling

Compute-related resources granted by core-year and GPU-year allocations require research groups to submit what are referred to as *jobs* to a *scheduler*. A job is a combination of a computer program (an application) and a list of resources that the application is expected to use. The **scheduler** is a program that calculates the priority of each job submitted and provides the needed resources based on the priority of each job and the available resources.

The scheduler uses prioritization algorithms to meet the allocation targets of all groups and it is based on a research group's recent usage of the system as compared to their allocated usage on that system. The past of the allocation period is taken into account but the most weight is put on recent usage (or non-usage). The point of this is to allow a research group that matches their actual usage with their allocated amounts to operate roughly continuously at that level. This smooths resource usage over time across all groups and resources, allowing for it to be theoretically possible for all research groups to hit their allocation targets.

Consequences of overusing a CPU or GPU allocation

If you have jobs waiting to run, and competing demand is low enough, then the scheduler may allow more of your jobs to run than your target level. The only consequence of this is that succeeding jobs of yours *may* have lower priority for a time while the scheduler prioritizes other groups which were below their target. You are not prevented from submitting or running new jobs, and the average of your usage over time should still be close to your target, that is, your allocation.

It is even possible that you could end a month or even a year having run more work than your allocation would seem to allow, although this is unlikely given the demand on our resources.

Reference GPU Units (RGUs)

The performance of GPUs has dramatically increased in the recent years and continues to do so. Until RAC 2023 we treated all GPUs as equivalent to each other for allocation purposes. This caused problems both in the allocation process and while running jobs, so in the 2024 RAC year we introduced the *reference GPU unit*, or **RGU**, to rank all GPU models in production and alleviate these problems. Since the 2025 RAC year we also have to deal with new complexity involving **multi-instance GPU technology**.

Because roughly half of our users primarily use single-precision floating-point operations (**FP32** (https://en.wikipedia.org/wiki/Single-precision_floating-point_format)), the other half use half-precision floating-point operations (**FP16** (https://en.wikipedia.org/wiki/Half-precision_floating-point_format)), and a significant portion of all users are constrained by the amount of memory on the GPU, we chose the following evaluation criteria and corresponding weights to rank the different GPU models:

Evaluation Criterion	Weight
FP32 score (with dense matrices on regular GPU cores)	40%
FP16 score (with dense matrices on <i>Tensor cores</i> (https://www.techspot.com/article/2049-what-are-tensor-cores/%7C))	40%
GPU memory score	20%

We currently use the NVidia **A100-40gb** GPU as the reference model and assign it an RGU value of 4.0 for historical reasons. We define its FP16 performance, FP32 performance, and memory size each as 1.0. Multiplying the percentages in the above table by 4.0 yields the following coefficients and RGU values for other models:

RGU scores for whole GPU models

	FP32 score	FP16 score	Memory score	Combined score	Available		Allocatable
Coefficient:	1.6	1.6	0.8	(RGU)	Now	2026	RAC 2026
H100-80gb	3.44	3.17	2.0	12.2	Yes	Yes	Yes
A100-80gb	1.00	1.00	2.0	4.8	?	?	No
A100-40gb	1.00	1.00	1.0	4.0	Yes	Yes	Yes
V100-32gb	0.81	0.40	0.8	2.6	No	No	No
V100-16gb	0.81	0.40	0.4	2.2	No	?	No
T4-16gb	0.42	0.21	0.4	1.3	No	No	No
P100-16gb	0.48	0.03	0.4	1.1	No	No	No
P100-12gb	0.48	0.03	0.3	1.0	No	No	No

With the 2025 **infrastructure renewal**, it will become possible to schedule a fraction of a GPU using **multi-instance GPU** technology. Different jobs, potentially belonging to different users, can run on the same GPU at the same time. Following **NVidia's terminology** (<https://docs.nvidia.com/datacenter/tesla/mig-user-guide/#terminology>), a fraction of a GPU allocated to a single job is called a *GPU instance*, also sometimes called a *MIG instance*.

The following table lists the GPU models and instances that can be selected in the CCDB form for RAC 2026. RGU values for GPU instances have been estimated from whole-GPU performance numbers and the fraction of the GPU which comprises the instance.

GPU models and instances available for RAC 2026

Model or instance	Fraction of GPU	RGU
A100-40gb	Whole GPU $\Rightarrow$ 100%	4.0
A100-1g.5gb	$\max(1\text{g}/7\text{g}, 5\text{GB}/40\text{GB}) \Rightarrow 14\%$	0.6
A100-2g.10gb	$\max(2\text{g}/7\text{g}, 10\text{GB}/40\text{GB}) \Rightarrow 28\%$	1.1
A100-3g.20gb	$\max(3\text{g}/7\text{g}, 20\text{GB}/40\text{GB}) \Rightarrow 50\%$	2.0
H100-80gb	Whole GPU $\Rightarrow$ 100%	12.2
H100-1g.10gb	$\max(1\text{g}/7\text{g}, 40\text{GB}/80\text{GB}) \Rightarrow 14\%$	1.7
H100-2g.20gb	$\max(2\text{g}/7\text{g}, 40\text{GB}/80\text{GB}) \Rightarrow 28\%$	3.5
H100-3g.40gb	$\max(3\text{g}/7\text{g}, 40\text{GB}/80\text{GB}) \Rightarrow 50\%$	6.1

Note: a GPU instance of profile **1g** is worth 1/7 of an A100 or H100 GPU. The case of **3g** takes into consideration the extra amount of memory per **g**. To simplify things for users, the **4g** profiles are not available on the clusters.

Choosing GPU models for your project

The relative scores in the above table should give you a hint on the models to choose. Here is an example with the extremes:

- If your applications are doing primarily FP32 operations, an A100-40gb GPU is expected to be twice as fast as a P100-12gb GPU, but the recorded usage will be 4 times the resources. Consequently, for an equal amount of RGUs, P100-12gb GPUs should allow you to run double the computations.
- If your applications (typically AI-related) are doing primarily FP16 operations (including mixed precision operations or using other floating-point formats (https://en.wikipedia.org/wiki/Bfloat16_floating-point_format)), using an A100-40gb will result in getting evaluated as using 4x the resources of a P100-12gb, but it is capable of computing ~30x the calculations for the same amount of time, which would allow you to complete ~7.5x the computations.

RAC awards hold RGU values constant

- During the Resource Allocation Competition (RAC), any proposal asking for GPUs must specify the preferred GPU model for the project. Then, in the CCDB form, the amount of reference GPU units (RGUs) will automatically be calculated from the requested amount of gpu-years per year of project.
 - For example, if you select the *narval-gpu* resource and request 13 gpu-years of the model A100-40gb, the corresponding amount of RGUs would be $13 * 4.0 = 52$. The RAC committee would then allocate up to 52 RGUs, depending on the proposal score. If your allocation must be moved to a different cluster, the committee will allocate gpu-years at that cluster so as to keep the amount of RGUs the same.

Detailed effect of resource usage on priority

The overarching principle governing the calculation of priority on our national clusters is that compute-based jobs are considered in the calculation based on the resources that others are prevented from using and not on the resources actually used.

The most common example of unused cores contributing to a priority calculation occurs when a submitted job requests multiple cores but uses fewer cores than requested when run. The usage that will affect the priority of future jobs is the number of cores requested, not the number of cores the application actually used. This is because the unused cores were unavailable to others to use during the job.

Another common case is when a job requests memory beyond what is associated with the cores requested. If a cluster that has 4GB of memory associated with each core receives a job request for only a single core but 8GB of memory, then the job will be deemed to have used two cores. This is because other researchers were effectively prevented from using the second core because there was no memory available for it.

Cores equivalent used by the scheduler

A core equivalent is a bundle made up of a single core and some amount of associated memory. In other words, a core equivalent is a core plus the amount of memory considered to be associated with each core on a given system.

On most of our clusters we define a core-equivalent to be 4GB and a core, since this is the memory-to-core ratio in the most common node type. Jobs are charged in terms of core-equivalent usage at the rate of 4 GB per core, as explained above. See Figure 1.

Allocation target tracking is straightforward when requests to use resources on the clusters are made entirely of core and memory amounts that can be portioned only into complete equivalent cores. Things become more complicated when jobs request portions of a core equivalent because it is possible to have many points counted against a research group's allocation, even when they are using only portions of core equivalents. In practice, the method used by the Alliance to account for system usage solves problems about fairness and perceptions of fairness but unfortunately the method is not initially intuitive.

Research groups are charged for the maximum number of core equivalents they take from the resources. Assuming a core equivalent of 1 core and 4GB of memory:

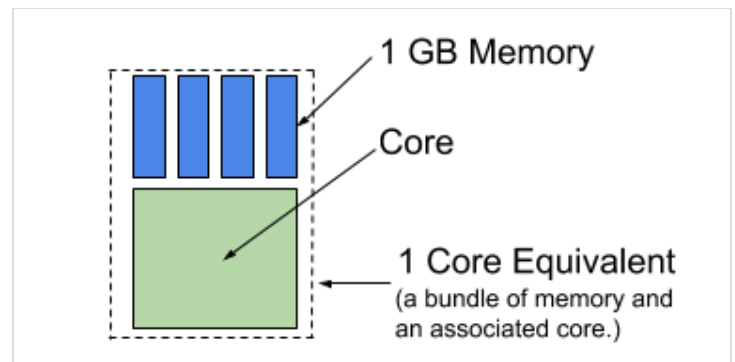


Figure 1 - Core equivalent diagram.

- Research groups using more cores than memory (above the 1 core/4GB memory ratio) will be charged by cores. For example, a research group requesting two cores and 2GB per core for a total of 4 GB of memory. The request requires 2 core equivalents worth of cores but only one bundle for memory. This job request will be counted as 2 core equivalents when priority is calculated. See Figure 2.

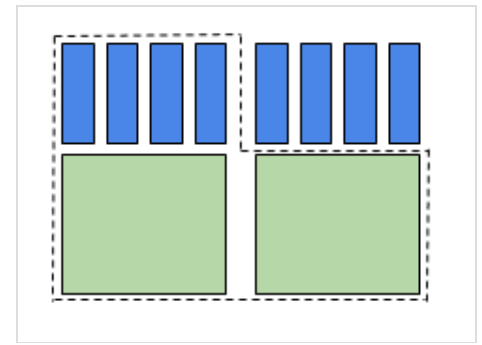


Figure 2 - Two core equivalents.

- Research groups using more memory than the 1 core/4GB ratio will be charged by memory. For example, a research group requests two cores and 5GB per core for a total of 10 GB of memory. The request requires 2.5 core equivalents worth of memory, but only two bundles for cores. This job request will be counted as 2.5 core equivalents when priority is calculated. See Figure 3.

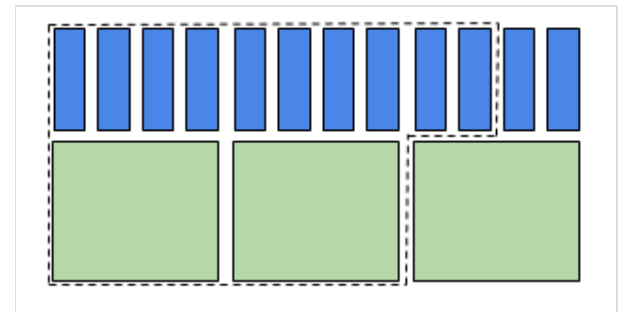


Figure 3 - 2.5 core equivalents.

Reference GPU unit equivalent used by the scheduler

Use of GPUs and their associated resources follow the same principles as already described for core equivalents, except that a reference GPU unit (RGU) is added to the bundle alongside multiple cores and memory. This means that the accounting for GPU-based allocation targets must include the RGU. Similar to how the point system was used above when considering resource use as an expression of the concept of core equivalence, we use a similar point system here as an expression of RGU equivalence.

Research groups are charged for the maximum number of RGU-core-memory bundles they use. Assuming a fictive bundle of 1 RGU, 3 cores, and 4 GB of memory:

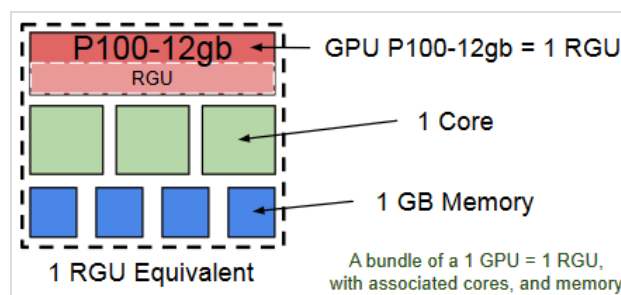


Figure 4 - RGU equivalent diagram.

- Research groups using more RGUs than cores or memory per RGU-core-memory bundle will be charged by RGU. For example, a research group requests 2 P100-12gb GPUs (1 RGU each), 3 cores, and 4 GB of memory. The request is for 2 bundles worth of RGUs, but only one bundle for memory and cores. This job request will be counted as 2 RGU equivalents when the research group's priority is calculated.

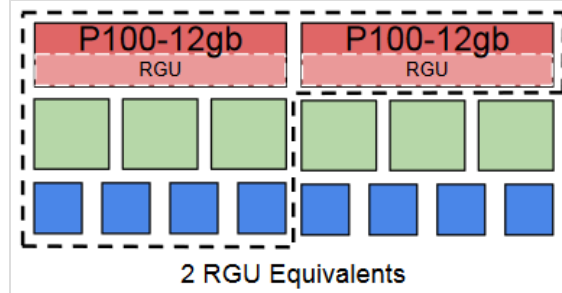


Figure 5 - Two RGU equivalents.

- Research groups using more cores than RGUs or memory per RGU-core-memory bundle will be charged by core. For example, a researcher requests 1 RGU, 5 cores, and 5 GB of memory. The request is for 1.66 bundles worth of cores, but only one bundle for RGUs and 1.25 bundles for memory. This job request will be counted as 1.66 RGU equivalents when the research group's priority is calculated.

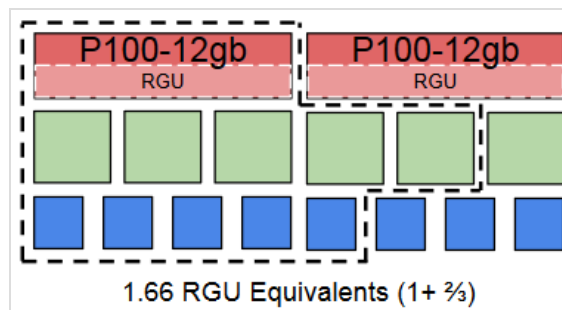


Figure 6 - 1.66 RGU equivalents, based on cores.

- Research groups using more memory than RGUs or cores per RGU-core-memory bundle will be charged by memory. For example, a researcher requests 1 RGU, 2 cores, and 6 GB of memory. The request is for 1.5 bundles worth of memory, but only one bundle for GPUs and 0.66 bundle for cores. This job request will be counted as 1.5 RGU equivalents when the research group's priority is calculated.

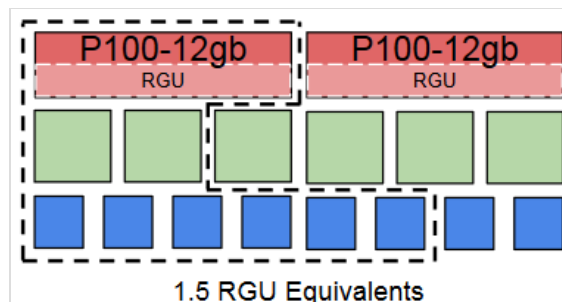


Figure 7 - 1.5 RGU equivalents, based on memory.

- On the same fictive cluster, a bundle with one V100-32gb GPU, 7.8 CPU cores and 10.4 GB of memory is

worth 2.6 RGU equivalents:

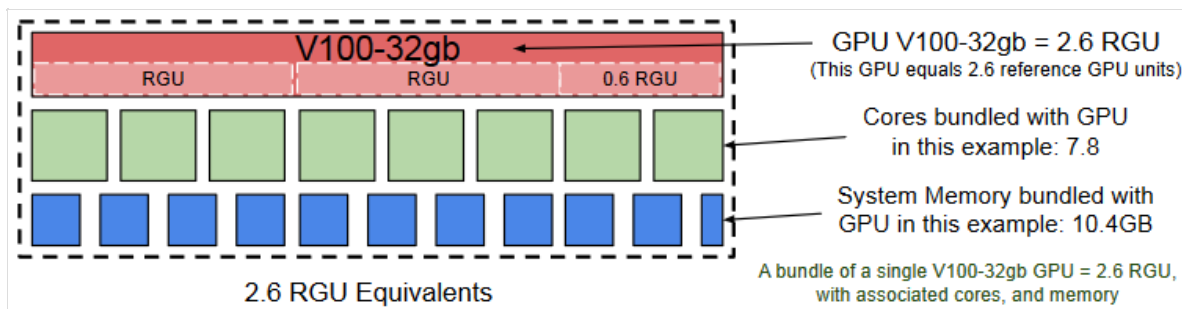


Figure 8 - 2.6 RGU equivalents, based on the V100-32gb GPU.

- On the same fictive cluster, a bundle with one A100-40gb GPU, 12 CPU cores and 16 GB of memory is worth 4.0 RGU equivalents:

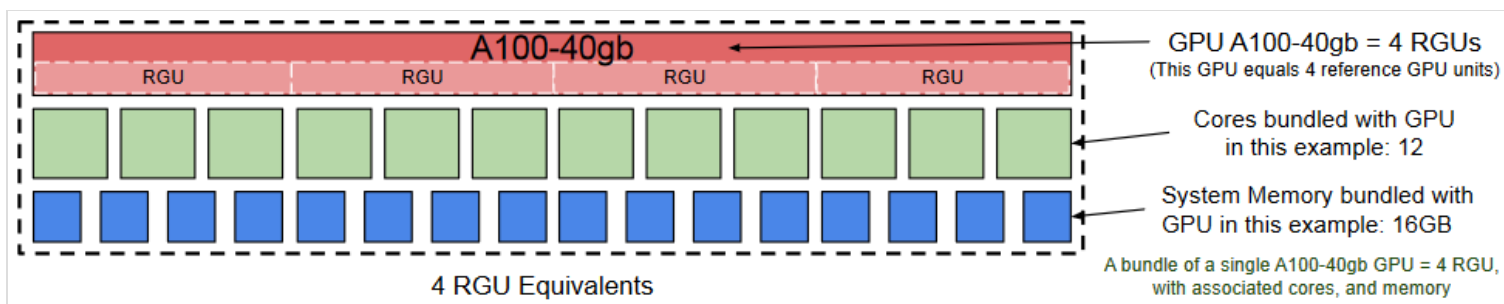


Figure 9 - 4.0 RGU equivalents, based on the A100-40gb GPU.

Ratios in bundles

Alliance systems have the following RGU-core-memory bundle characteristics (only one ratio per cluster):

Cluster	Cores per RGU	Memory per RGU (GB)
Fir	0.98	23.6
Narval	3.00	31.1
Nibi	1.15	20.5
Rorqual	1.31	10.2
Trillium	1.97	15.4

And the following GPU-core-memory bundle characteristics:

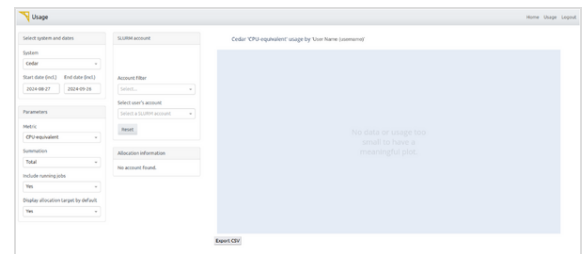
Cluster	Model or instance	RGU per GPU	Bundle per GPU	Recommended per GPU
Fir	H100-80gb	12.2	12 cores, 288 GB	12 cores, 280 GB
	H100-1g.10gb	1.74	1.7 cores, 41 GB	1 core, 35 GB
	H100-2g.20gb	3.48	3.4 cores, 82 GB	3 cores, 70 GB
	H100-3g.40gb	6.1	6 cores, 144 GB	6 cores, 140 GB
Narval	A100-40gb	4.0	12 cores, 124.5 GB	12 cores, 124 GB
	A100-1g.5gb	0.57	1.7 cores, 17.7 GB	1 core, 15 GB
	A100-2g.10gb	1.14	3.4 cores, 35.4 GB	3 cores, 31 GB
	A100-3g.20gb	2.0	6.0 cores, 62.2 GB	6 cores, 62 GB
	A100-4g.20gb	2.3	6.9 cores, 71.5 GB	6 cores, 62 GB
Nibi	H100-80gb	12.2	14 cores, 250 GB	14 cores, 250 GB
	H100-1g.10gb	1.74	2 cores, 35.7 GB	2 cores, 31 GB
	H100-2g.20gb	3.48	4 cores, 71.4 GB	4 cores, 62 GB
	H100-3g.40gb	6.1	7 cores, 125 GB	6 cores, 124 GB
Rorqual	H100-80gb	12.2	16 cores, 124.5 GB	16 cores, 124 GB
	H100-1g.10gb	1.74	2.3 cores, 17.7 GB	2 cores, 15 GB
	H100-2g.20gb	3.48	4.5 cores, 35.4 GB	4 cores, 31 GB
	H100-3g.40gb	6.1	8 cores, 62.2 GB	8 cores, 62 GB
Trillium	H100-80gb	12.2	24 cores, 188 GB	24 cores, 188 GB

Note: While the scheduler will compute the priority based on the usage calculated with the above bundles, users requesting multiple GPUs per node also have to take into account the physical ratios.

Viewing resource usage in the portal

portal.alliancecan.ca/slurm (<https://portal.alliancecan.ca/slurm>) provides an interface for exploring time-series data about jobs on our national clusters. The page contains a figure that can display several usage metrics. When you first log in to the site, the figure will display CPU days on an arbitrary cluster across all project accounts that you have access to. If you have no usage on that cluster, the figure will contain the text *No Data or usage too small to have a meaningful plot*. The data appearing in the figure can be modified by control panels along the left margin of the page. There are three panels:

- Select system and dates
- Parameters
- SLURM account



Usage portal landing view. (Click on the image for a larger version.)

Displaying a specified account

If you have access to more than one **Slurm account**, the *Select user's account* pull-down menu of the *SLURM account* panel lets you select which project account will be displayed in the figure window. If the *Select user's account* is left empty the figure will display all of your usage across accounts on the specified cluster during the

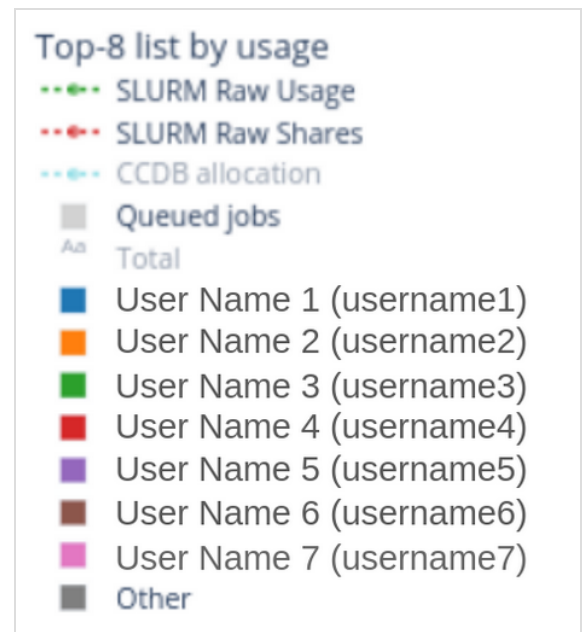
selected time period. The *Select user's account* pull-down menu is populated by a list of all the accounts that have job records on the selected cluster during the selected time interval. Other accounts that you have access to but do not have usage on the selected cluster during the selected time interval will also appear in the pull-down menu but will be grayed out and not selectable as they would not generate a figure. When you select a single project account the figure is updated and the summary panel titled *Allocation Information* is populated with details of the project account. The height of each bar in the histogram figure corresponds to the metric for that day (e.g. CPU-equivalent days) across all users in the account on the system. The top seven users are displayed in unique colors stacked on top of the summed metric for all other users in gray. You can navigate the figure using **Plotly** (<https://plotly.com/graphing-libraries/>) tools (zoom, pan, etc.) whose icons appear at the top-right when you hover your mouse over the figure window. You can also use the legend on the right-hand side to manipulate the figure. Single-clicking an item will toggle the item's presence in the figure, and double-clicking the item will toggle off or on all the other items in the figure.



Usage display of a specified account

Options in the figure legend

The legend of the usage figure provides display options. Specifically, additional variables can be enable or disabled from the figure legend. Beyond displaying the colour affiliation of each user displayed in the usage figure, the legend provides access to displaying *SLURM Raw Usage*, *SLURM Raw Shares*, *CCDB allocation*, the resources pending for *Queued jobs* and the daily *Total*. *SLURM Raw Usage* and *SLURM Raw Shares* is obtained from a poll of *sshare* for each account on the clusters. *CCDB allocation* is the account profile at CCDB representation of the *SLURM Raw Shares*. *Queued jobs* is a metric that represents the quantity of resources belonging to jobs that are pending in the job queue and is represented with narrow gray bars for each day in the figure window. *Total* adds text at the top of each bar indicating the daily total of the metric across users for the day. By single clicking any of the items in the legend the specific item is toggled in the figure window. By double clicking any of the items in the legend all other items in the figure are toggled on or off.

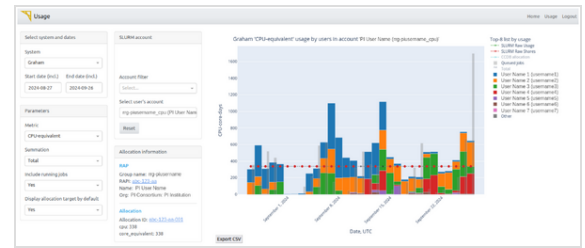


Usage figure legend

Displaying the allocation target and queued resources

When a single account has been selected for display, the *SLURM Raw Shares* is shown as a horizontal red line. It can be turned off or on with the *Display allocation target by default* item in the *Parameters* panel, or by clicking on *SLURM Raw Shares* in the legend to the right of the figure.

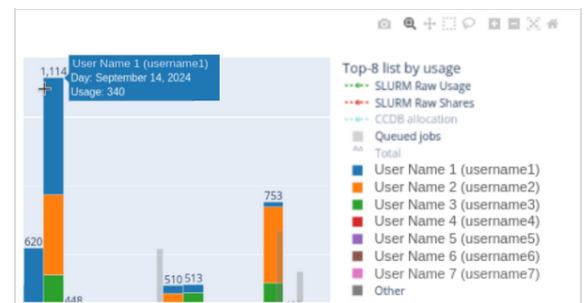
You can toggle the display of the *Queued jobs* metric, which presents a sum of all resources in pending jobs at each time point, by clicking on the words *Queued jobs* in the legend to the right of the figure.



Allocation target and queued resources displayed on usage figure

Mouse hover over the figure window

Native Plotly interactive figure options are made available at the top right of the figure window when the mouse pointer hovers over the figure. Icons for *Download plot as a png*, *Zoom*, *Pan*, *Box Select*, *Lasso Select*, *Zoom in*, *Zoom out*, *Autoscale* and *Reset axes* allow for interactive navigation of the figure selection and scale. When hovering over bar items in the figure window, a hover text appears providing the *User Name*, *Day* and *Usage* quantity of the specific item under the pointer (note that this returns the usage quantity for the specific user not the sum across users for the day).

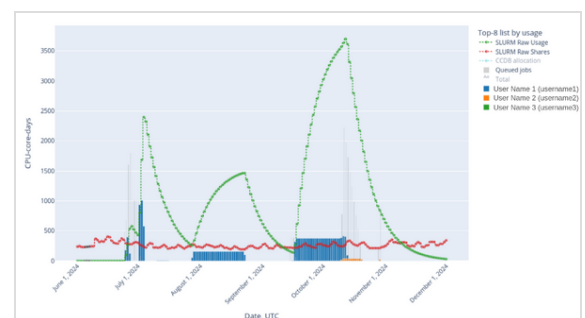


Additional figure options when hover the mouse over the figure window

Default SLURM Raw Shares and the SLURM Raw Usage

The SLURM Raw Shares of an allocation *rrg-** or *rpp-** account is a straight line that corresponds to the account's resource allocation on the cluster. For default accounts the SLURM Raw Shares are dynamic over time based on the number of active accounts on the cluster. Plotting the SLURM Raw Shares of a default account on a specific cluster is an easy way of determining the expected usage that can be achieved by a default account on a given cluster.

The SLURM Raw Usage is a metric that the scheduling software uses to determine the priority of accounts. The SLURM Raw Usage is the cumulative sum of the accounts usage in billing units plus a half life decay period. Plotting the account's SLURM Raw Usage is a convenient way to assess how past



Display of default account usage, including SLURM Raw Shares and SLURM Raw Usage

usage influences the account's priority over time. A good rule of thumb in these figures is that if the SLURM Raw Usage is at 10 times the SLURM Raw Shares then the account's usage is at par with its target share (e.g. the usage rate that the schedule will try to maintain for the account).

Selecting a specific cluster and time interval

The figure shows your usage for a single cluster over a specified time interval. The *System* pull-down menu contains entries for each of the currently active national clusters that use Slurm as a scheduler. You can use the "Start date (incl.)" and "End date (incl.)" fields in the "Select system and dates" panel to change the time interval displayed in the figure. It will include all jobs on the specified cluster that were in a running (R) or pending (PD) state during the time interval, including both the start and end date. Selecting an end date in the future will display the *projection* of currently running and pending jobs for their requested duration into the future.

Select a specific cluster and time interval

Displaying usage over an extended time period into the future

If you select an end time after the present time, the figure will have a transparent red area overlaid on the future time labelled *Projection*. In this projection period, each job is assumed to run to the time limit requested for it. For queued resources, the projection supposes that each pending job starts at the beginning of the projected time (that is, right now) and runs until its requested time limit. This is not intended to be a forecast of actual future events!



Display usage over an extended period into the future

Metrics, summation, and running jobs

Use the *Metric* pull-down control in the *Parameters* panel to select from the following metrics: CPU, CPU-equivalent, RGU, RGU-equivalent, Memory, Billing, gpu, and all specific GPU models available on the selected cluster.

The *Summation* pull-down allows you to switch between the daily *Total* and *Running total*. If you select *Total*, each bar of the histogram represents the total usage in that one day. If you select "Running total", each bar represents the sum of that day's usage and all previous days back to the beginning of the time interval. If the *Allocation Target* is displayed, it is similarly adjusted to show the running total of the target usage. See the next section for more.

If you set *Include Running jobs* to *No*, the figure shows only data from records of completed jobs. If you set it to *Yes* it includes data from running jobs too.

Parameters

Metric
CPU-equivalent

Summation
Total

Include running jobs
Yes

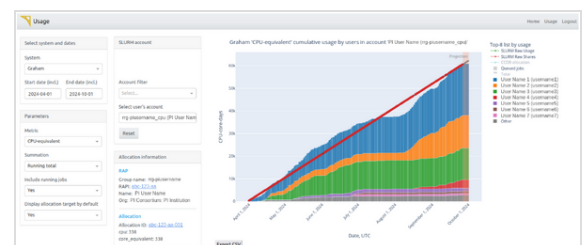
Display allocation target by default
Yes

Parameters of the usage series histogram

Display of the running total of account usage

When displaying the running total of the usage for a single account along with the *Allocation target* the usage histogram displays how an account deviates from its target share over the period displayed. The values in this view are the cumulative sum across days from "total" summation view for both the usage and allocation target. When an account is submitting jobs that request more than the account's target share, it is expected that the usage cumulative sum will oscillate above and below the target share cumulative sum if the scheduler is managing fair share properly.

Because the scheduler uses a decay period for the impact of past usage, a good interval to use to inspect the scheduler's performance in maintaining the account's fair share is to display the past 30 days.



Display the running total of account usage

Viewing resource usage in CCDB

Information on the usage of compute resources by your groups can be found by logging into the CCDB and navigating to *My Account > View Group Usage*.

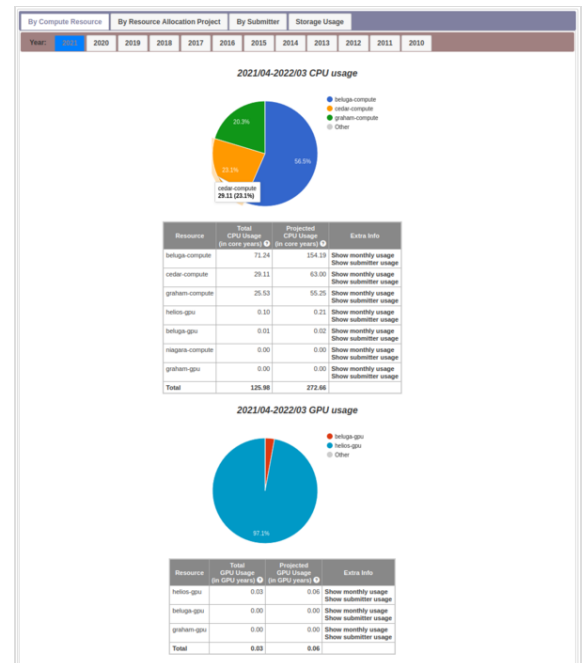


Navigation to *View Group Usage*

CPU and GPU core year values are calculated based on the quantity of the resources allocated to jobs on the clusters. It is important to note that the values summarized in these pages do not represent core-equivalent measures such that, in the case of large memory jobs, the usage values will not match the cluster scheduler's representation of the account usage.

The first tab bar offers these options:

- By Compute Resource:** cluster on which jobs are submitted;
- By Resource Allocation Project:** projects to which jobs are submitted;
- By Submitter:** user that submits the jobs;
- Storage usage** is discussed in [Storage and file management](#).

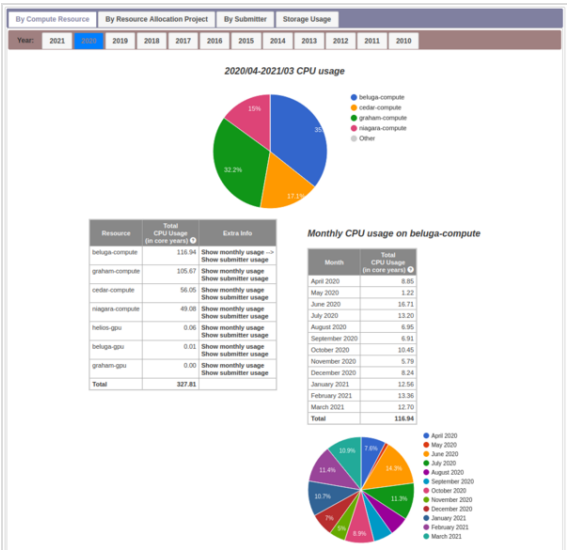


CPU and GPU usage by compute resource

Usage by compute resource

This view shows the usage of compute resources per cluster used by groups owned by you or of which you are a member for the current allocation year starting April 1st. The tables contain the total usage to date as well as the projected usage to the end of the current allocation period.

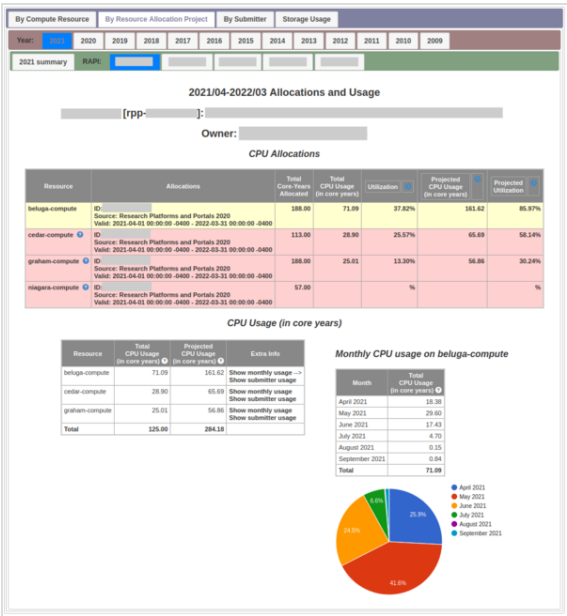
From the *Extra Info* column of the usage table *Show monthly usage* can be clicked to display a further breakdown of the usage by month for the specific cluster row in the table. By clicking *Show submitter usage*, a similar breakdown is displayed for the specific users submitting the jobs on the cluster.



Usage by compute resource with monthly breakdown

Usage by resource allocation project

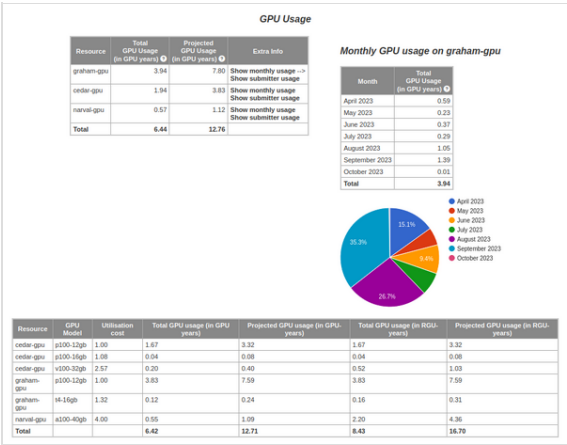
Under this tab, a third tag bar displays the RAPIs (Resource Allocation Project Identifiers) for the selected allocation year. The tables contain detailed information for each allocation project and the resources used by the projects on all of the clusters. The top of the page summarizes information such as the account name (e.g. def-, rrg- or rpp-*, etc.), the project title and ownership, as well as allocation and usage summaries.



Usage by Resource Allocation Project with monthly breakdown

GPU usage and Reference GPU Units (RGUs)

For resource allocation projects that have GPU usage, the table is broken down into usage on various GPU models and measured in RGUs.



GPU usage summary with Reference GPU Unit (RGU) breakdown table.

Usage by submitter

Usage can also be displayed grouped by the users that submitted jobs from within the resource allocation projects (group accounts). The view shows the usage for each user aggregated across systems. Selecting from the list of users will display that user’s usage broken down by cluster. Like the group summaries, these user summaries can then be broken down monthly by clicking the Show monthly usage link of the Extra Info column of the CPU/GPU Usage (in core/GPU years) table for the specific Resource row.

